
COMPTE RENDU DE PROJET

Méthodes numériques appliquées à l'IA

Sommaire

1. **Introduction**
2. **Architecture de la bibliothèque de RL**
 - 2.1. Organisation du code
 - 2.2. Structure des classes
 - 2.3. Choix techniques et justification
3. **Implémentation des algorithmes de renforcement**
 - 3.1. ϵ -Greedy
 - 3.2. Upper Confidence Bound (UCB)
 - 3.3. Bandit Gradient
 - 3.4 Policy Iteration
 - 3.5 Value Iteration
 - 3.6 Q-learning
4. **Études de cas et expérimentations**
 - 4.1. SimpleRunner : Recommandation de films
 - 4.2. YouTubeRunner : Flux de vidéos et préférences évolutives
 - 4.3. TicTacToeRunner : Apprentissage par jeu
5. **Fonction de récompense et apprentissage**
 - 5.1. Modélisation des récompenses
 - 5.2. Adaptation aux environnements
6. **Conclusion et remarques**

1. Introduction

Ce projet s'inscrit dans le cadre du cours *Méthodes numériques pour l'intelligence artificielle* et vise à concevoir, en Java, une bibliothèque d'algorithmes de *Reinforcement Learning* (RL). L'objectif principal est d'implémenter différentes stratégies d'apprentissage automatique permettant à un agent d'interagir avec des environnements décrits de manière formelle, en apprenant à maximiser une récompense.

L'approche adoptée repose sur l'utilisation de spécifications formelles comme représentation de l'environnement. Celles-ci sont interprétées via l'outil ProB, qui permet de générer dynamiquement l'espace d'états et de transitions. À partir de cette base, nous avons développé une bibliothèque modulaire intégrant plusieurs algorithmes classiques de RL, notamment **ϵ -Greedy**, **UCB**, et **Bandit Gradient**, **Value Iteration**, **Policy Iteration** et **Q-learning**, chacun testé sur différents cas d'étude.

Ce document constitue le rapport final et détaille l'ensemble des travaux réalisés, des choix techniques aux résultats expérimentaux sur des environnements variés, allant de la recommandation de vidéos (Simple et Youtube) à des jeux stratégiques comme le morpion (TicTacToe).

2. Architecture de la bibliothèque de RL

2.1. Organisation du code

La bibliothèque est organisée de la manière suivante :

- Le code source est situé dans `src/main/java/fr/polytech/mnia/`
- Le code est divisé en plusieurs sous-packages :
 - **agents** : contient les différentes implémentations d'agents de Reinforcement Learning.
 - **graphs** : destiné à la visualisation.
 - Les fichiers Java restants (**App.java, et les différents runners**) jouent un rôle de test/démonstration.
- D'autres répertoires comme **ressources** et **docs** sont présents pour la documentation et les fichiers annexes (donnés au préalable).

Cette organisation reflète une séparation claire des responsabilités : **agents d'apprentissage**, **visualisation**, et **exécution/test**.

2.2. Structure des classes

- **Classe Agent.java** : Interface définissant les comportements de base des agents. Les autres classes comme `QlearningAgent.java` ou `EpsilonGreedyAgent.java` en héritent.
- **Agents spécialisés** :
 - `QlearningAgent`, `PolicyIterationAgent`, `ValueIterationAgent` implémentent des algorithmes de RL.
 - `BanditGradientAgent`, `UCBAgent`, `EpsilonGreedyAgent` gèrent des problèmes de bandits manchots.
- **Graphique (Visualization.java)** : permet d'afficher graphiquement les résultats d'apprentissage des agents, notamment l'évolution des probabilités d'action et des récompenses cumulées. Elle offre également des outils pour comparer les performances de plusieurs agents et visualiser leurs préférences d'action. (voir remarque)
- **App** : La classe `App` est le **point d'entrée principal** de l'application. Elle sert à exécuter un scénario de test en instanciant un objet d'un des *Runner* définis dans le projet. Pour exécuter un scénario, il suffit de choisir le numéro du scénario souhaité (ex: 2 pour `YoutubeRunner`)
- **Runners** : responsables de l'exécution de scénarios d'apprentissage, avec différents environnements (`TicTacToeRunner`, `SimpleRunner` et `YoutubeRunner`). Chaque classe exécute les agents appropriés sur le même scénario et compare les résultats finaux en affichant à chaque fois les graphes appropriés grâce à la classe `visualization` importée. La fonction du reward étant définie dans le même fichier.

2.3. Choix techniques et justification

- **Modularité** : Chaque agent est dans un fichier distinct, favorisant la lisibilité et la praticité pour les modifications à apporter.
- **Nommage clair** : Les noms des fichiers/classes sont explicites (ex: `PolicyIterationAgent.java`), ce qui facilite la compréhension du code.
- **Runner séparé** : L'existence de plusieurs classes étendant "Runner" permet de tester différents environnements sans toucher au cœur du code des agents.
- **Fonctions de reward** : directement implémentées dans les runners.
- **Visualisation intégrée** : Présence d'un module graphique, utile pour observer le comportement des agents.

3. Implémentation des algorithmes de renforcement

L'implémentation de ces agents est expliquée dans le code source en commentaires détaillés. Ces algorithmes sont implémentés à partir de ceux qu'on a vus en cours.

4. Études de cas et expérimentations

4.1. SimpleRunner : Recommandation de films

Ce runner est un test comparatif d'algorithmes de bandits manchots sur un modèle B spécifié dans SimpleRL.mch. Il compare les trois agents : ϵ -Greedy, UCB et Bandit Gradient. L'environnement modélise un utilisateur qui aime les films A et B, mais pas C, et chaque agent doit apprendre cela en interagissant avec l'environnement pendant 500 étapes. À chaque étape, l'agent :

1. Choisis un film (action) à recommander.
2. Observe une récompense (1 si film aimé, 0 sinon).
3. Met à jour sa stratégie d'après ce retour.
4. Stocke la récompense pour analyse.

Interprétation des résultats :

→ ϵ -Greedy

Avec Q-values : A = 1, B = 1, C = 0., l'agent choisi quasiment uniquement le film A (exploration faible). Donc il a appris que A et B donnent des récompenses, mais a peu exploré B. C'est le comportement attendu avec $\epsilon=0.1$: 90% exploitation, 10% exploration.

→ UCB

Q-values : A = 1.0, B = 1.0, C = 0.

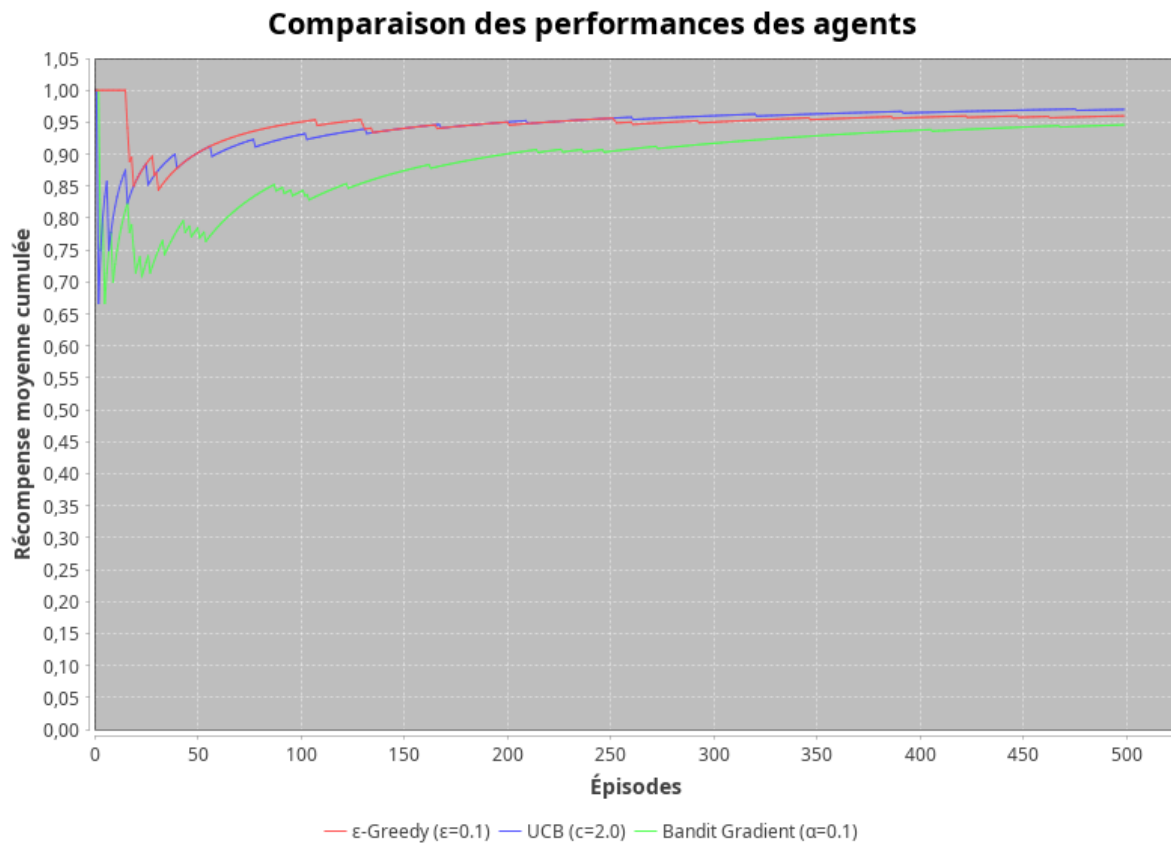
L'agent est censé alterner entre A et B, explorant mieux que ϵ -Greedy. Grace au bonus d'exploration (logarithmique), il a évité le piège de l'exploitation prématurée. Plus équilibré dans l'utilisation de A et B.

→ Bandit Gradient

Politique finale (probabilités) : A =54%, B =45%, C =2% (varie en fonction du test bien évidemment, mais les résultats restent similaires).

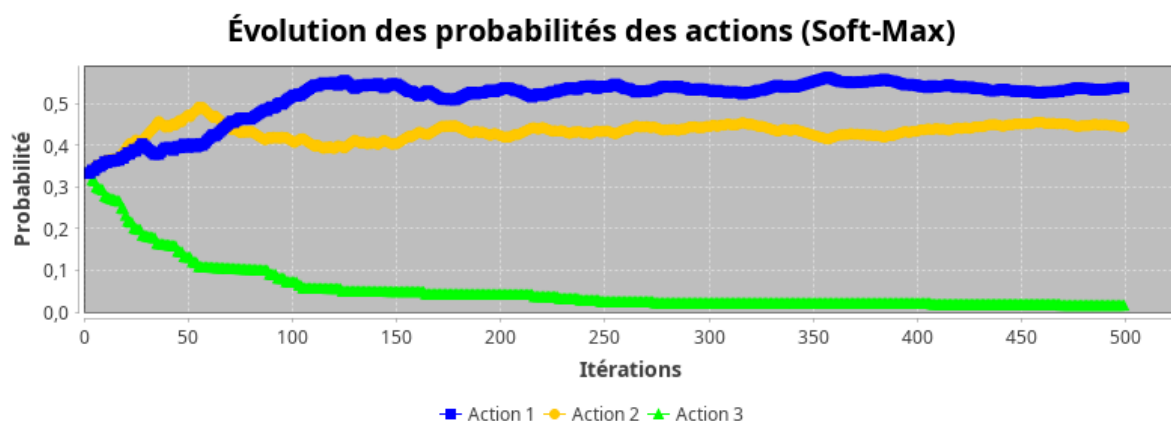
A convergé vers une distribution quasi-optimale en attribuant très peu de poids à C. Contrairement aux autres, cet agent n'est pas déterministe mais apprend une politique stochastique (via Softmax).

❖ Graphiques :



Ce graphique compare les trois stratégies. On voit que ϵ -Greedy (rouge) converge rapidement vers la récompense maximale (exploite très vite la meilleure option). Très efficace ici car l'environnement est statique et simple. UCB (ici en bleu) apprend un peu plus lentement, mais atteint aussi une très bonne performance. Utile quand on veut un bon compromis entre exploration/exploitation. En revanche, en vert le Bandit Gradient qui est plus lent à converger, et reste légèrement en dessous des deux autres. Toutefois, il maintient une certaine exploration (surtout au début) et arrive à avoir une bonne convergence.

★ Intéressons nous un peu plus au Bandit Gradient!

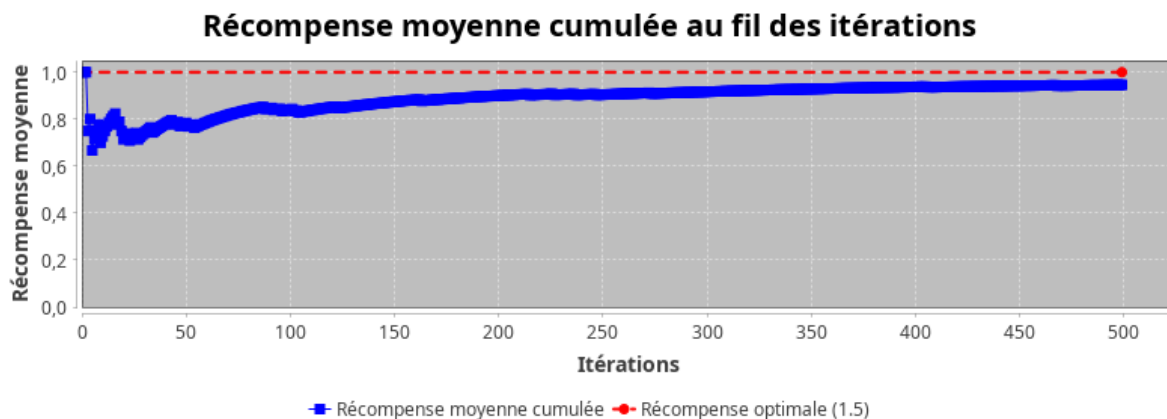


Ce graphique montre l'évolution des probabilités de sélection des trois films par le Bandit Gradient :

- Action 1 (Film A) est clairement identifiée comme la meilleure (probabilité de 0.6+).
- Action 2 (Film B) reste également sélectionnée avec une proba non négligeable (0.35), ce qui est cohérent car B rapporte aussi une récompense.

- Action 3 (Film C) est progressivement abandonnée (proba proche de 0), car il ne rapporte aucune récompense.

Cela montre que la politique softmax de l'agent est bien apprise.



Ce graphique montre l'évolution de la récompense moyenne cumulée pour l'agent Bandit Gradient ($\alpha=0.1$) :

- La courbe bleue converge progressivement vers la récompense maximale de 1.
- Ce qui signifie que l'agent apprend efficacement à privilégier les bras rentables (films A et B), même si l'apprentissage est un peu plus lent que d'autres stratégies.

En conclusion, tous les agents atteignent une bonne performance, mais dans ce cas statique et déterministe, ϵ -Greedy et UCB sont les plus rapides à converger. Le Bandit Gradient, bien qu'un peu plus lent, apprend à privilégier les bons films et maintient une politique adaptative. Ce comportement est exactement ce qu'on attend d'un système de recommandation intelligent dans un environnement statique.*

4.2. YoutubeRunner : Système de recommandation évolutif de vidéos Youtube

La classe YoutubeRunner simule un système de recommandation de vidéos YouTube en utilisant les mêmes agents utilisés. Elle repose sur un modèle B non déterministe (YouTube.mch) où la récompense est proportionnelle à la durée de visionnage d'une vidéo, reflétant ainsi l'intérêt de l'utilisateur. Le runner enregistre les récompenses obtenues et la fréquence de sélection de chaque vidéo, permettant une analyse comparative des stratégies d'apprentissage face à des préférences utilisateur dynamiques.

Interprétation des résultats :

(nous interprétons ici des résultats obtenus dû au lancement d'un test quelconque, mais cela va de soi que les conclusions restent les mêmes).

→ ϵ -Greedy ($\epsilon=0.1$)

```
--- Résultats finaux pour :  $\epsilon$ -Greedy ( $\epsilon=0.1$ ) ---
```

Vidéo	Valeur	Sélections	Pourcentage	
Tutoriel_Python	0,9820	125	2,50	%
Vlog_de_voyage	0,1992	126	2,52	%
Musique_populaire	0,6115	240	4,80	%
Gaming	1,0000	4509	90,18	%

Cet agent privilégie l'exploitation de l'option la plus rentable connue, avec une exploration limitée. Il a rapidement identifié "Gaming" comme la meilleure option, ce qui explique sa sélection majoritaire. Pareillement pour les musiques populaires qu'il choisit souvent. En revanche, il n'ose pas exploiter d'autres options qui pourraient être au final intéressantes.

→ Agent UCB (c=2.0)

```

--- Résultats finaux pour : UCB (c=2.0) ---

```

Vidéo	Valeur	Sélections	Pourcentage
Tutoriel_Python	0,9955	2300	46,00 %
Vlog_de_voyage	0,3222	54	1,08 %
Musique_populaire	0,6591	165	3,30 %
Gaming	1,0000	2481	49,62 %

L'agent UCB équilibre exploration et exploitation en tenant compte de l'incertitude des estimations. Il a su s'adapter aux changements de préférences, notamment en augmentant la sélection de "Tutoriel_Python" lorsque son intérêt a augmenté. Cette capacité d'adaptation le rend efficace dans des environnements dynamiques.

→ Agent Bandit Gradient ($\alpha=0.1$)

```

--- Résultats finaux pour : Bandit Gradient ( $\alpha=0.1$ ) ---

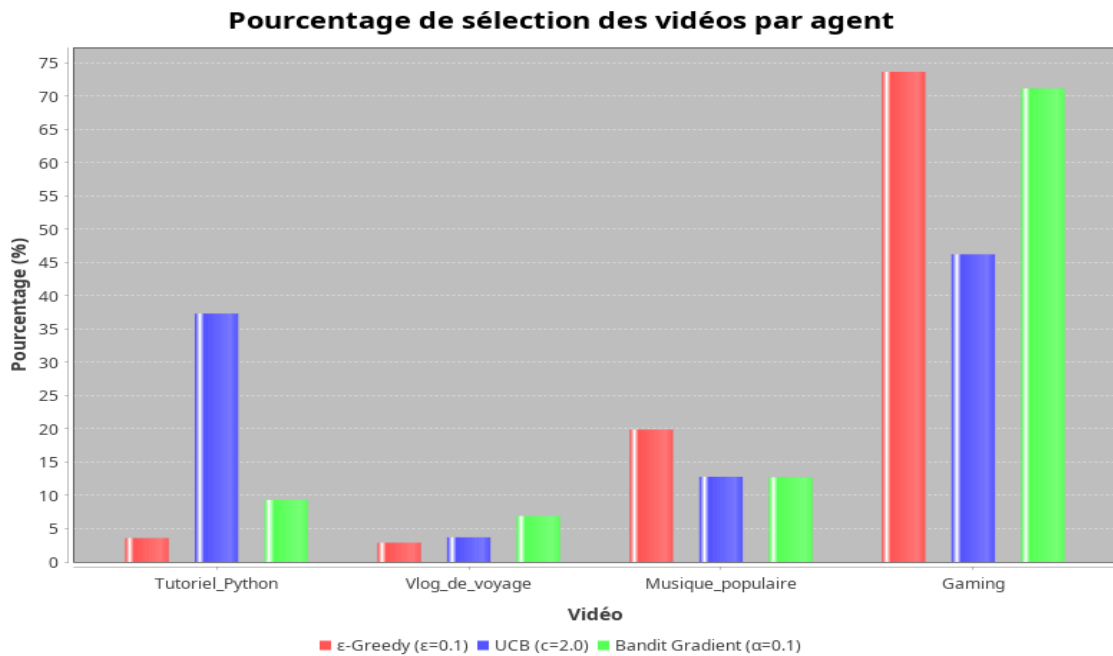
```

Vidéo	Valeur	Sélections	Pourcentage
Tutoriel_Python	0,0124	170	3,40 %
Vlog_de_voyage	0,0016	86	1,72 %
Musique_populaire	0,0027	136	2,72 %
Gaming	0,9833	4608	92,16 %

Cet agent ajuste ses préférences en fonction des récompenses obtenues, sans estimer explicitement les valeurs des actions. Il a fortement favorisé "Gaming", mais a montré une adaptation limitée aux changements de préférences, au début vers "Musiques populaires" ensuite vers "Python_tutoriels".

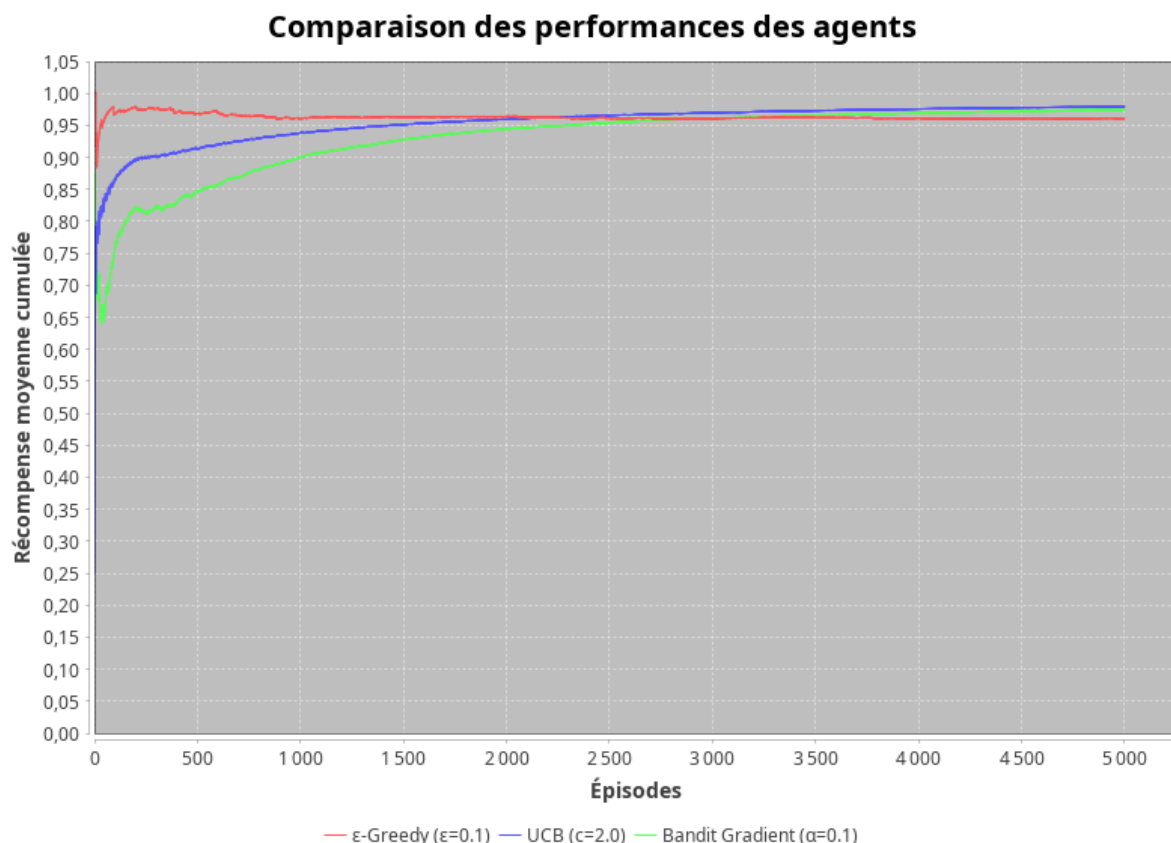
En conclusion, parmi les trois agents, les agents UCB et Bandit Gradient ont montré la meilleure capacité d'adaptation aux préférences changeantes de l'utilisateur, en équilibrant efficacement exploration et exploitation. L'agent ϵ -Greedy, bien qu'efficace dans des environnements stationnaires, peut être moins performant lorsque les préférences évoluent.

❖ Graphiques :



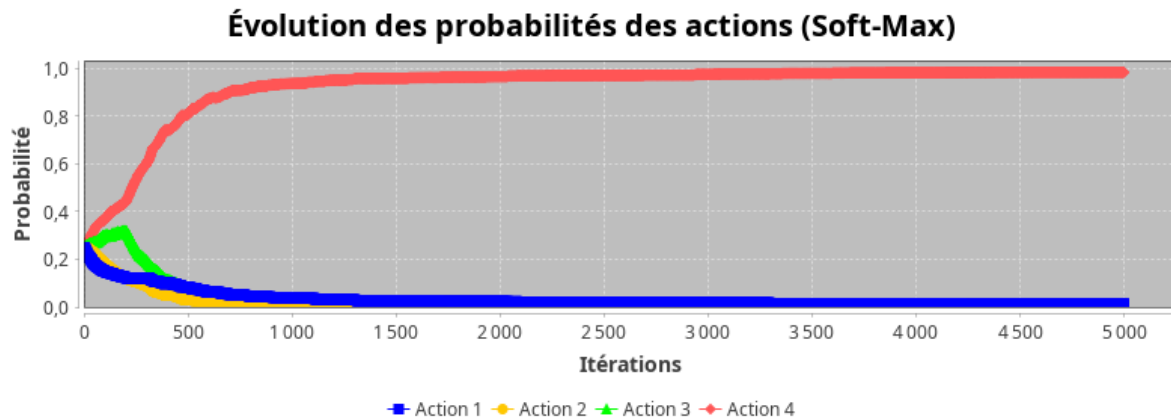
Ce graphique montre le **pourcentage de sélection de chaque vidéo** par les 3 agents après 1000 itérations.

- Gaming est la vidéo la plus attractive pour tous les agents, signe qu'elle offre la meilleure récompense moyenne (temps de visionnage élevé).
- L'agent ϵ -Greedy exploite fortement la meilleure vidéo, avec peu d'exploration.
- UCB garde une bonne exploration, surtout sur *Tutoriel_Python*, qui semble devenir bon au fil du temps.
- Bandit Gradient montre une bonne adaptation, en concentrant la probabilité sur *Gaming*. Au début, il s'intéresse aux musiques populaires avant de basculer sur l'exploration des tutoriels pythons qui semblent de plus en plus intéressants et c'est exactement le comportement attendu.



- ϵ -Greedy est très performant ici car l'environnement a une récompense stable pour *Gaming*, qu'il apprend vite à exploiter.
- UCB explore plus, donc met plus de temps à converger mais reste robuste.
- Bandit Gradient s'adapte lentement. Cela peut être dû à la dynamique changeante des préférences utilisateur

★ Intéressons nous encore une fois au Bandit Gradient!



Le graphique montre comment les probabilités de sélection des vidéos évoluent avec le temps avec le bandit gradient (en bleu [Tutoriel_Python](#); en jaune [Vlog_de_voyage](#), en vert [Musique_Populaire](#); [Gaming](#) en rouge).

- Action 4 (rouge, Gaming) devient dominante (> 85% à la fin).
- Action 3 (vert, musique) est choisie au début mais est très vite "abandonnée".
- Les vlogs de voyage sont très peu explorés dès le début.
- Enfin, on voit que l'agent explore plus les tutoriels python que les musiques populaires, s'adaptant aux nouvelles préférences mais finit par laisser tomber cette option comme le reste, préférant toujours gaming à tout le reste.

4.3. TicTacToeRunner : Apprentissage par jeu

Ce runner teste différents agents d'apprentissage par renforcement dans un environnement déterministe : le jeu du TicTacToe. L'agent apprend à jouer contre un adversaire aléatoire (Joueur 0 dans notre implémentation) via TicTacToe.mch implémenté dans ProB. Les récompenses sont : +1 en cas de victoire, 0 en cas de match nul ou jeu non terminé, -1 en cas de défaite.

Nous avons implémenté et testé trois approches : **Policy Iteration**, **Value Iteration** et **Qlearning**

La comparaison de ces agents se fait dans le TicTacToeRunner dans la méthode `compareAgents()` qui lance l'apprentissage des 3 agents, les compare avec leurs récompenses moyennes accumulées et lance la version interactive du TicTacToe contre l'agent ayant eu les meilleures performances (le joueur joue Joueur 0, l'agent joue Joueur 1).

Pour chaque agent nous avons pris les paramètres classiques suivants : $\epsilon = 0.1$, $\gamma = 0.9$, $\alpha = 0.1$ (voir `App.java` pour les changer).

Note : Pour le TicTacToe, nous avons implémenté une nouvelle fonction `initialiseAlealInitialState()` dans `Runner` pour avoir des états initiaux différents d'un épisode à un autre -> cela facilitait l'exploration en début d'apprentissage.

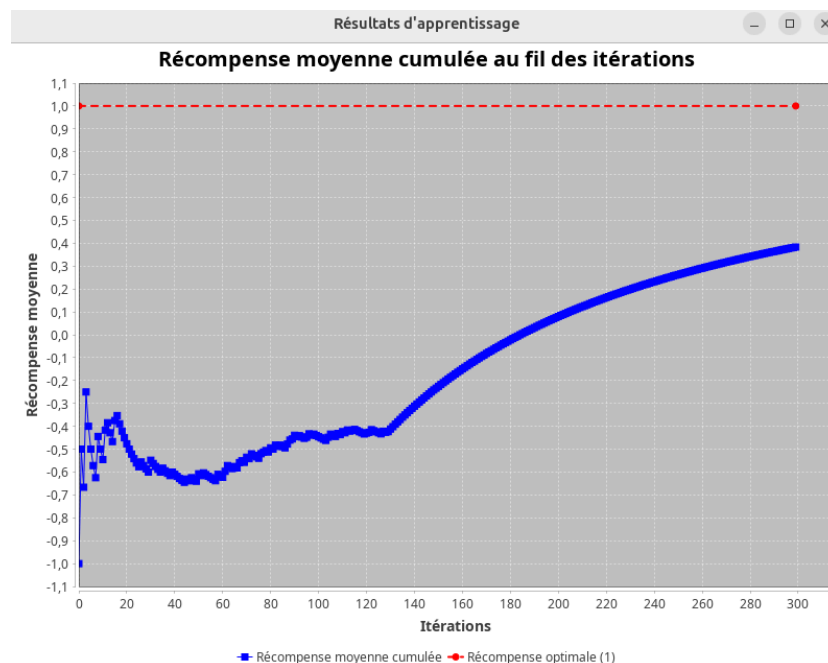
Interprétation des résultats :

→ Policy Iteration :

Dans notre implémentation de Policy Iteration, nous avons choisi de faire une longue phase d'exploration, puis de l'exploitation, car l'environnement du TicTacToe est stable et nous avons besoin de beaucoup de transitions et d'états visités avant de pouvoir commencer l'apprentissage.

Le nombre d'états possibles étant environ de 19000, nous avons choisi de commencer l'exploitation au bout de 2000 états/transitions parcouru (ajusté aussi par rapport au nombre d'épisodes joué, car si trop peu d'épisodes -> pas d'exploitation car pas assez de données)

Note : L'exécution de Policy Iteration Agent lors de la phase d'apprentissage est très lente, nous aurions sûrement pu améliorer ses performances en terme d'efficacité de calcul en triant au préalable la liste d'état visités ainsi que les transitions et récompenses associées (en organisant un système peu similaire aux caches memoires), mais nous n'en avons pas eu le temps.



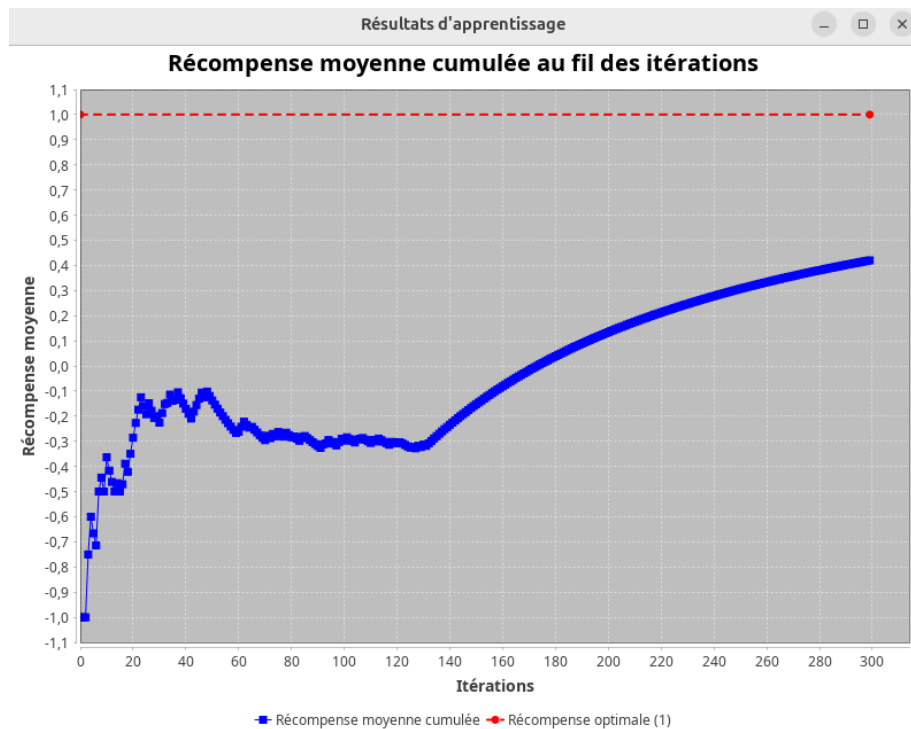
Policy Iteration Agent sur 300 épisodes

Ici on voit bien la différence entre le moment de l'exploration (environ durant 130 épisodes ici) et la phase d'exploitation où l'agent ne perd quasiment plus du tout.

→ Value Iteration :

Dans notre implémentation de Value Iteration, comme pour Policy Iteration, nous avons choisi de faire une longue phase d'exploration, puis de l'exploitation. Et ce pour les mêmes raisons que citées précédemment : environnement stable et de nombreux états possibles.

Contrairement à Policy Iteration, l'exécution de Value Iteration est plus rapide car les calculs sont moins coûteux.

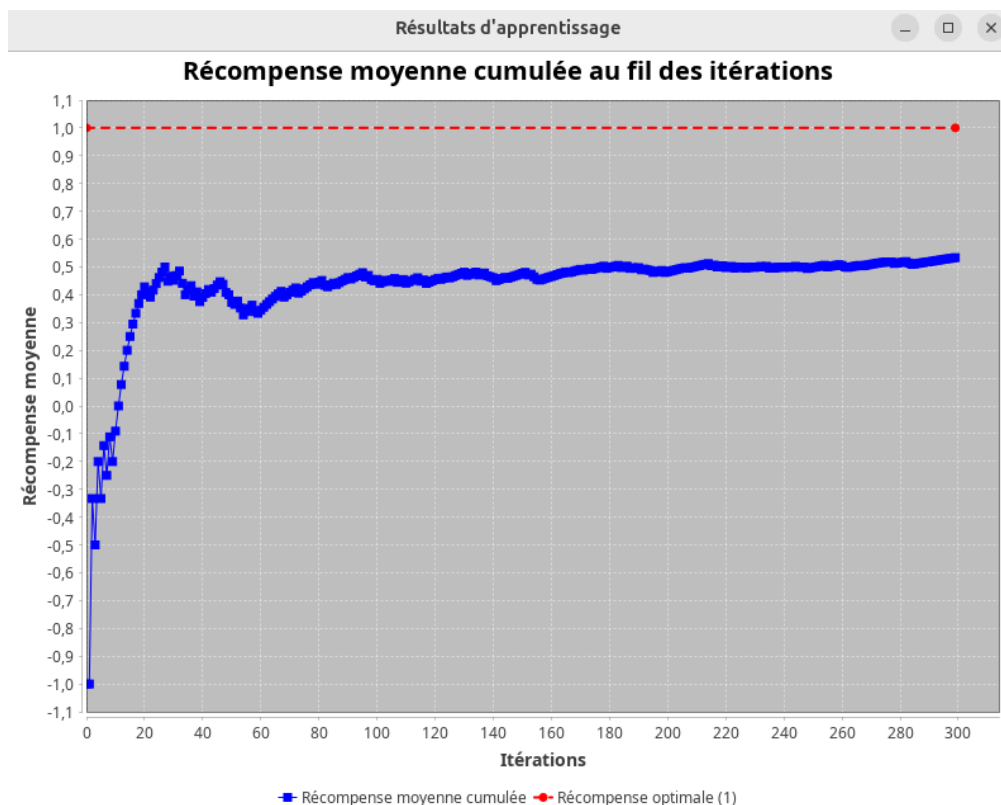


ValueIterationAgent sur 300 épisodes

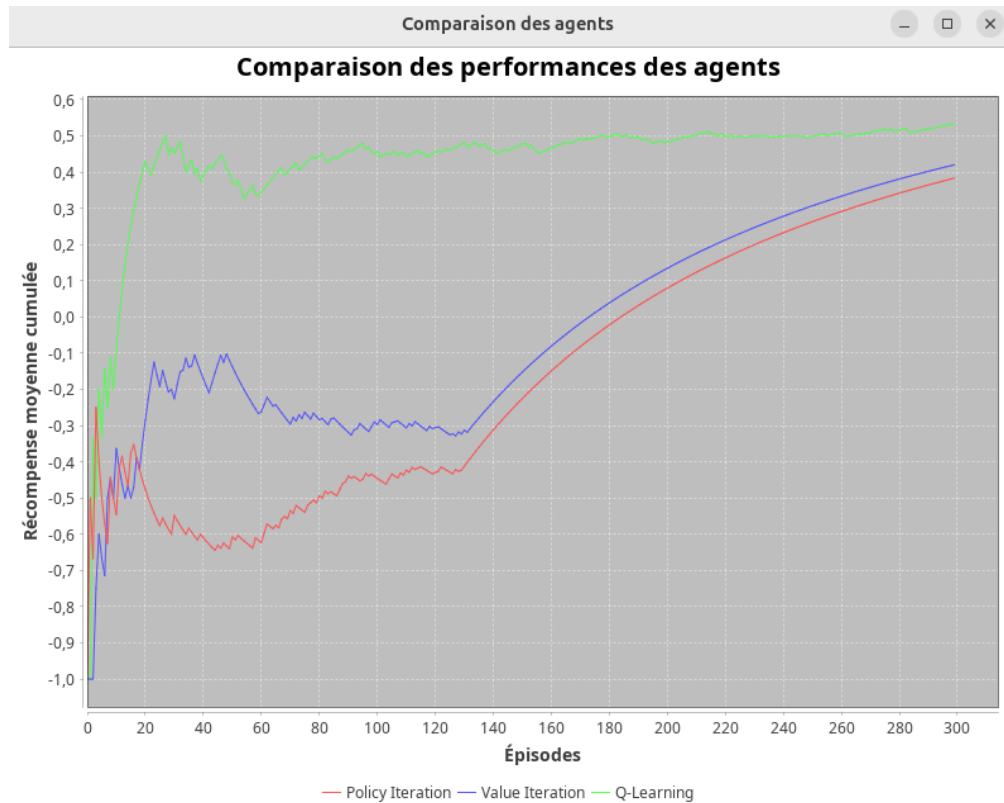
On retrouve encore bien nettement la séparation entre phase d'exploration et d'exploitation. Encore autour de 130 épisodes on voit la rupture après laquelle l'agent ne perd presque plus. Cette similarité s'explique par le fait que nous avons mis le même seuil d'expériences à acquérir que pour PolicyIteration (à savoir 2000 états visités).

→ Qlearning :

L'agent du Qlearning était plus simple à implémenter, l'agrémentation de la Qtable était plus rapide que les listes d'états et de transitions nécessaires aux algorithmes nécessitant le connaissant du modèle (MDP) comme PolicyIteration et ValueIteration. Qlearning est un algorithme modèle-free ce qui donne également une convergence plus rapide.



Ici on observe une convergence beaucoup plus rapide que pour les 2 agents précédents, cependant il semble stagner autour d'une récompense à 0.5, tandis que les 2 autres semblent tendre vers une valeur plus haute, ce qui est cohérent avec la théorie. (Voir à la fin du 4.3 pour une série de 500 épisodes par agent)



- Qlearning converge rapidement vers 0.5, ce qui est cohérent puisque c'est un modèle-free, il est donc plus intéressant lorsqu'on entraîne assez peu l'agent (ici 300 épisodes)
- Policy Iteration explore pendant un certain temps avant de converger vers 0.39, mais on devine qu'avec plus d'entraînement ses performances dépassent celle du Qlearning.
- De même, Value Iteration explore au début puis exploite et tend rapidement vers 0.41.

Note : En théorie, Policy Iteration, même si ses calculs sont plus coûteux, devrait converger plus vite que Value Iteration. Peut être aurions nous dû lancer la phase d'exploitation de Policy Iteration plus tôt pour obtenir de meilleurs résultats, au lieu de la calquer sur Value Iteration.

★ Partie interactive du Joueur contre l'agent entraîné :

```
>>> Meilleur agent : Q-Learning avec moyenne 0.5333333333333333

=== Partie interactive contre l'agent entraîné ===
1 1|1 2|1 3
---+---+---
2 1|2 2|2 3
---+---+---
3 1|3 2|3 3
C'est votre tour. Entrez les coordonnées x y (1 à 3) :
2 2
  |  |
---+---+---
  | 0 |
---+---+---
  |  |

Coup de l'Agent
1  |  |
---+---+---
  | 0 |
---+---+---
  |  |

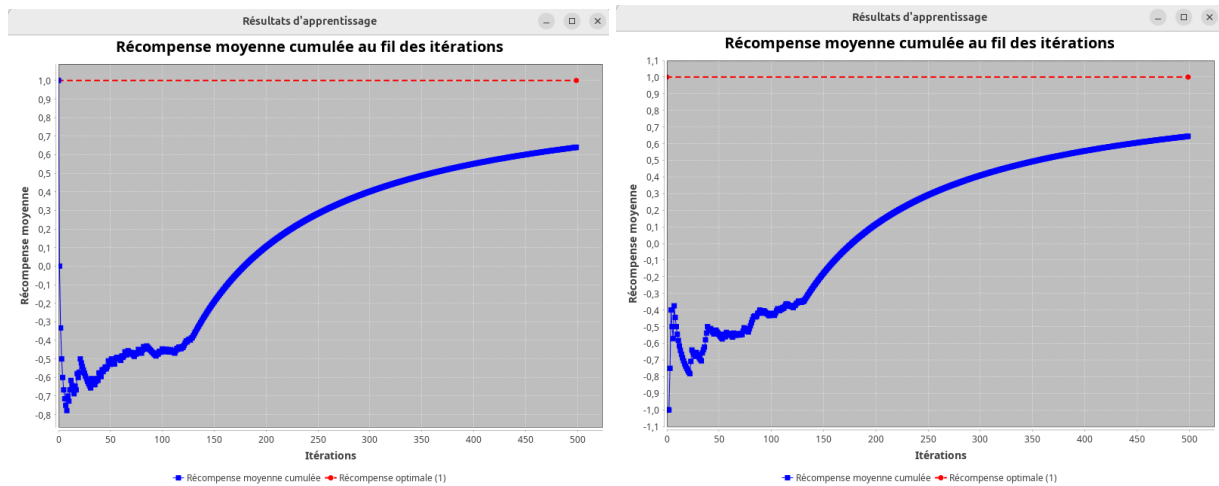
C'est votre tour. Entrez les coordonnées x y (1 à 3) :
2 1
1  |  |
---+---+---
0 | 0 |
---+---+---
  |  |

Coup de l'Agent
1  |  |
---+---+---
0 | 0 |
---+---+---
1  |  |
C'est votre tour. Entrez les coordonnées x y (1 à 3) :
2 3
1  |  |
---+---+---
0 | 0 | 0
---+---+---
1  |  |
Vous avez gagné !
Voulez-vous rejouer ? (o/n) o
```

On constate que l'agent ne parvient pas vraiment à contrer le joueur. Plusieurs raisons sont possibles :

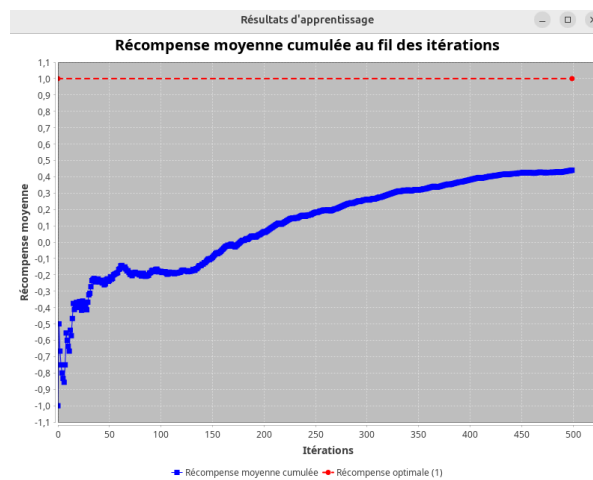
- Manque d'entraînement
- L'agent ne s'est entraîné que contre un joueur aléatoire et ça n'a pas suffi
- Erreur dans l'application de l'algorithme

Courbes pour 500 épisodes par agent :

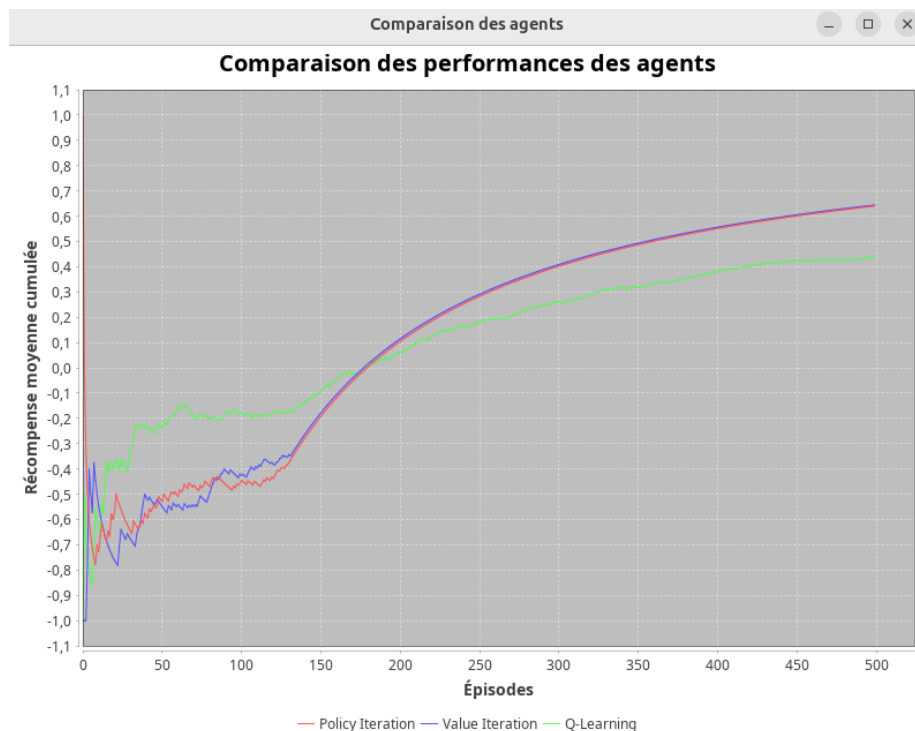


Policy Iteration Agent sur 500 épisodes

Value Iteration Agent sur 500 épisodes



Qlearning Agent sur 500 épisodes



Comme on le devinait, plus le nombre d'épisodes est grand, plus QlearningAgent se fait dépasser par Policy Iteration Agent et Value Iteration Agent qui ici évoluent de manière analogue.

5. Fonction de récompense et apprentissage

5.1. Modélisation des récompenses

➤ Dans SimpleRunner :

Le modèle SimpleRL.mch est déterministe. L'utilisateur aime les films A et B, pas C. La récompense est donc modélisée simplement dans le code par :

```
// String res = getState().eval("res").toString();  
// double reward = res.equals("OK") ? 1.0 : 0.0;
```

Si l'utilisateur aime le film, res = OK, donc récompense = 1 Sinon, récompense = 0

➤ Dans YoutubeRunner :

Le modèle YouTube.mch est non déterministe et plus réaliste. La récompense dépend de la durée de visionnage d'une vidéo :

```
// String rawDuration = getState().eval("duration").toString();  
// double reward = parseDuration(rawDuration); // ex "25" devient 0.25
```

Ici, la fonction de parsing sert à transformer une chaîne de caractères représentant une durée (comme "25" ou "duration = 100") en une valeur numérique normalisée entre 0.0 et 1.0, utilisable comme récompense dans un algorithme de bandit manchot, elle return duration / 100.0.

Ici, la récompense est une valeur continue entre 0 et 1, représentant l'engagement de l'utilisateur.

➤ Dans TicTacToeRunner :

Le modèle TicTacToe.mch est déterministe et il n'y a que 3 issues :

- Victoire : +1
- Match nul ou partie en cours : 0
- Défaite : -1

On utilise la méthode **getState().eval("win(1)").toString()** pour évaluer la victoire de Joueur 1 (ou de Joueur0).

5.2. Adaptation aux environnements

➤ Dans SimpleRunner :

L'environnement est fixe: l'utilisateur ne change pas d'avis. Les agents apprennent des préférences stables car on attend que A et B soient choisis de plus en plus, C de moins en moins. Tous les algorithmes convergent relativement rapidement.

➤ Dans YoutubeRunner :

L'environnement est dynamique : les préférences de l'utilisateur changent au cours du temps .Les algorithmes doivent donc adapter leur politique face à ces variations.

Par exemple, un agent doit abandonner progressivement la musique au profit des tutos pythons.

➤ Dans TicTacToeRunner :

L'environnement est fixe, les états possibles sont finis et les conditions de victoires ne change pas. On peut considérer que l'environnement est stable, c'est pourquoi nous avons choisi une longue phase d'exploration dans Policy Iteration et Value Iteration, suivant de la phase d'exploitation.

6. Conclusion et remarques

Ce projet nous a permis de concevoir une bibliothèque d'agents de reinforcement learning polyvalente, capable de s'adapter à plusieurs types d'environnements. En abordant à la fois des cas simples et plus complexes comme le jeu de TicTacToe ou la recommandation personnalisée, nous avons pu expérimenter les avantages et les limites de chaque approche. Les agents itératifs (Value Iteration, Policy Iteration) se sont montrés très efficaces lorsque l'environnement était intégralement connu et stationnaire, tandis que les méthodes exploratoires comme Q-learning et les bandits manchots ont mieux répondu aux environnements partiellement observables ou aux contextes dynamiques.

Au-delà de l'implémentation technique, ce projet nous a permis de mieux comprendre les subtilités des compromis entre exploration et exploitation, et l'importance du choix de l'algorithme en fonction des caractéristiques de la situation.

Remarques :

Nous avons fait le choix d'utiliser ChatGPT pour l'implémentation de la classe de visualisation (Visualization.java). Étant donné que ce projet demandait un investissement conséquent en termes de logique algorithmique et de modélisation, nous avons considéré que le codage des graphes ne constituait pas une compétence pédagogique centrale attendue dans ce contexte. L'objectif principal étant de tester une bibliothèque d'algorithmes d'apprentissage par renforcement, nous avons jugé plus pertinent de nous concentrer sur cet aspect.

Par ailleurs, nous avons dû modifier la machine B du fichier YouTube.mch, car dans sa version initiale, la variable "duration" n'était pas accessible directement via `eval("duration")` dans le code Java. En effet, dans le modèle d'origine, "duration" était retournée comme valeur de sortie de l'opération `choose (duration <- choose(vv))`, mais n'était pas définie comme une variable d'état globale.

Pour contourner cette limitation et permettre un accès simplifié depuis notre code Java, nous avons :

- Ajouté "duration" comme variable d'état dans la section VARIABLES de la machine.
- Modifié l'opération choose pour qu'elle ne retourne plus de valeur, mais modifie directement la variable "duration" à chaque appel.

Grace à cette modification, nous avons pu utiliser la ligne `getState().eval("duration")` pour récupérer la durée de visionnage après chaque sélection de vidéo, ce qui simplifie considérablement l'intégration avec nos agents.